Generative AI Fundamentals

LLMs, Transformers, Embeddings, RAG, Prompting and Deployment

A short study guide used as the knowledge source for the RAG chatbot notebook.

1. What is Generative AI?

Generative AI (GenAI) refers to artificial intelligence systems that can create new content such as text, images, audio, video and code. Unlike traditional machine learning models that classify or predict from existing data, generative models learn the underlying patterns of their training data and use them to produce new, original outputs.

Popular examples include large language models (LLMs) such as GPT, Claude, Gemini, Llama and Qwen for text; diffusion models such as Stable Diffusion for images; and code assistants that write and explain programs. GenAI became mainstream in late 2022 with the release of ChatGPT, which showed that conversational AI could be useful to millions of non-technical users.

2. Large Language Models (LLMs)

A Large Language Model is a neural network trained on huge amounts of text to predict the next token in a sequence. By repeating this prediction step, the model can write essays, answer questions, translate languages and generate code. The word 'large' refers to the number of parameters, which ranges from hundreds of millions in small models to hundreds of billions in frontier models.

Training happens in stages. Pre-training teaches the model general language ability from web pages, books and code. Supervised fine-tuning (SFT) teaches it to follow instructions using curated examples. Reinforcement learning from human feedback (RLHF) or similar preference-tuning methods then align its answers with what people consider helpful, honest and harmless.

Instruction-tuned models, often named with the suffix 'Instruct' or 'Chat', are the ones normally used in chatbots. Small models such as Qwen2.5-0.5B, 1.5B and 3B can run on a laptop or a free Google Colab GPU, while larger models need powerful hardware or a hosted API.

3. The Transformer Architecture

Almost all modern LLMs are built on the Transformer architecture introduced in the 2017 paper 'Attention Is All You Need'. The key idea is self-attention, a mechanism that lets every token in the input look at every other token and decide how much each one matters for understanding the current word.

A Transformer is made of stacked layers. Each layer contains a multi-head self-attention block and a feed-forward network, connected with residual connections and layer normalization. Because attention processes all tokens in parallel, Transformers train much faster on GPUs than older recurrent networks such as LSTMs.

Text is first split into tokens, which are pieces of words. Each token is converted into a vector called an embedding. Positional information is added so the model knows the order of the words. The context window is the maximum number of tokens the model can read at once; modern models support from a few thousand to over a million tokens.

4. Embeddings and Vector Databases

An embedding is a list of numbers that represents the meaning of a piece of text. Sentences with similar meaning have embeddings that are close to each other in vector space. Models such as sentence-transformers/all-MiniLM-L6-v2 convert a sentence into a 384-dimensional vector and are widely used for semantic search.

A vector database stores embeddings and can quickly find the ones most similar to a query vector using measures such as cosine similarity. Common vector databases are Chroma, FAISS, Pinecone, Weaviate and Qdrant. Chroma is popular for learning projects because it is open source, simple to install and can run fully in memory.

5. Retrieval-Augmented Generation (RAG)

LLMs only know what they saw during training, so they can be out of date and may not know private company documents. They also sometimes hallucinate, meaning they produce confident but false statements. Retrieval-Augmented Generation (RAG) reduces these problems by giving the model relevant documents at question time.

A RAG pipeline has two phases. In the indexing phase, documents such as PDFs are loaded, split into small overlapping chunks, converted to embeddings and stored in a vector database. In the query phase, the user question is embedded, the top-k most similar chunks are retrieved, and these chunks are inserted into a prompt together with the question. The LLM then writes an answer grounded in that context.

Important RAG settings include chunk size (for example 500 to 1000 characters), chunk overlap (for example 10 to 20 percent of the chunk size), and the number of retrieved chunks k (usually 3 to 5). Chunks that are too small lose context, while chunks that are too large add noise and may exceed the context window.

RAG is usually cheaper and easier to maintain than fine-tuning because updating knowledge only requires re-indexing documents, not retraining the model. Source citations can also be shown to users so they can verify every answer.

6. Prompt Engineering and Generation Settings

Prompt engineering is the practice of writing instructions that get the best output from a model. Good prompts define the role of the assistant, give clear instructions, provide context, state the desired format and may include examples. Techniques include zero-shot prompting, few-shot prompting and chain-of-thought prompting, where the model is asked to reason step by step.

Temperature controls randomness during generation. A value near 0 makes the model pick the most likely token every time, giving focused and repeatable answers that suit factual question answering. Higher values such as 0.7 to 1.0 make the output more varied and creative. In Hugging Face Transformers, temperature only takes effect when do_sample is set to True.

Other common settings are top_p (nucleus sampling), which limits sampling to the smallest group of tokens whose probabilities add up to p; top_k, which limits sampling to the k most likely tokens; max_new_tokens, which caps the answer length; and repetition_penalty, which discourages the model from repeating itself.

7. Fine-Tuning and Parameter-Efficient Methods

Fine-tuning continues training a pre-trained model on a smaller, task-specific dataset so that it learns a particular style, format or domain. Full fine-tuning updates all weights and is expensive. Parameter-efficient fine-tuning (PEFT) methods such as LoRA (Low-Rank Adaptation) and QLoRA train only a small number of extra weights, which cuts memory use dramatically and allows large models to be adapted on a single GPU.

As a rule of thumb, use prompting first, add RAG when the model needs external or private knowledge, and use fine-tuning when you need consistent behavior, tone or structured output that prompting cannot achieve.

8. Agents, Tools and LangChain

An AI agent is an LLM that can decide which actions to take, call external tools such as web search, calculators, databases or APIs, observe the results and continue until the task is finished. Function calling lets the model return structured requests that the application executes.

LangChain is an open-source framework that provides building blocks for LLM applications: document loaders, text splitters, embedding wrappers, vector store integrations, prompt templates and output parsers. The LangChain Expression Language (LCEL) lets developers connect these pieces with the pipe operator to build chains. LangGraph extends this idea to stateful, multi-step agent workflows.

9. Evaluation, Risks and Responsible AI

Evaluating GenAI systems is difficult because many tasks have no single correct answer. Common approaches include benchmark datasets, human review, and using a strong LLM as a judge. For RAG systems, teams measure retrieval quality (are the right chunks found?), faithfulness (is the answer supported by the context?) and answer relevance. Frameworks such as RAGAS automate these checks.

Key risks include hallucinations, bias inherited from training data, leakage of private information, copyright concerns, prompt injection attacks and misuse for misinformation. Responsible deployment involves content filtering, human oversight, clear disclosure that content is AI-generated, privacy protection and continuous monitoring.

10. Applications of Generative AI

GenAI is used in customer-support chatbots, document question answering, code generation, content writing, translation, summarization, education and tutoring, healthcare documentation, data analysis, design and marketing. In education, a RAG chatbot built on college documents or course material can answer student questions instantly, while in business it can search policies, contracts and manuals.

Deployment options include notebooks for experiments, Streamlit or Gradio for quick web interfaces, FastAPI for production back-ends, and hosting on Hugging Face Spaces, Streamlit Community Cloud or cloud providers such as AWS, Azure and Google Cloud.